

MEMORIA DE PRACTICAS: ARDUINO

María Soriano Palacios

Microgrado en Electrónica y Automática Industrial

Práctica 1. Parpadeo de LED

En este proyecto haremos que se ilumine un LED en modo intermitente, de esta forma, al cambiar su estado entre alto y bajo alternativamente lo veremos parpadear continuamente. Para este esquema solo necesitaremos una salida digital, por lo que, como podemos ver, será el puerto 13, que suministrará energía intermitente al LED.

Descripción

En esta práctica se parte de un circuito básico de parpadeo de LED. El objetivo es hacer que un LED conectado a una salida digital del Arduino se encienda y se apague de forma intermitente. En el enunciado se indica que el LED original se conecta al pin digital 13 y que la velocidad de parpadeo se controlará mediante la función *delay*. Además, se utiliza el monitor serie para mostrar mensajes con el estado del LED. Posteriormente, modificaremos el circuito añadiendo dos LED más para construir un semáforo con un funcionamiento cíclico: primero se enciende el LED rojo durante 3 segundos, después el LED verde durante 3 segundos y finalmente el LED naranja durante 500 milisegundos.

¿Cómo se esta utilizando el monitor serie y delay?

El monitor serie se utiliza para **mostrar información textual** sobre lo que está ocurriendo en el programa **mientras se ejecuta**. En el código de ejemplo, se inicia la comunicación serie con la instrucción `'Serial.begin(9600)'` lo que configura la velocidad de transmisión a 9600 baudios. Después, dentro de la función `'loop'`, se emplea `'Serial.println("On")'` cuando el LED se enciende y `'Serial.println("Off")'` cuando el LED se apaga. De esta manera, en el monitor serie se puede observar un **registro** continuo del **estado del LED**, y comprobar así que el programa funciona correctamente.

En cuanto a la función `'delay'`, se utiliza para **detener temporalmente la ejecución** del programa durante el número de milisegundos indicado. En el ejemplo, se usa `'delay(500)'`, lo que significa que el programa espera 500 milisegundos, es decir, medio segundo.

Gracias a esto, el LED permanece encendido medio segundo y apagado medio segundo, produciendo el efecto de parpadeo. **Si se disminuye este valor**, el LED parpadeará **más rápido**; si se aumenta, parpadeará más lentamente. En la modificación del semáforo, esta función permitirá mantener cada LED encendido durante el tiempo requerido por el enunciado: **3000 ms para rojo, 3000 ms para verde y 500 ms para naranja**.

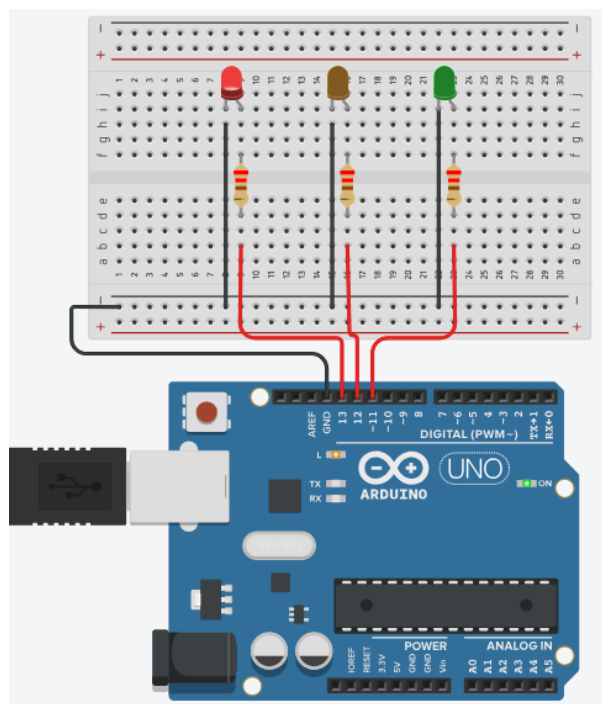
Modificación del diseño: Semáforo

Se ha modificado el circuito añadiendo dos LED adicionales para representar un semáforo: un LED rojo, un LED naranja y un LED verde. Cada LED se ha conectado a un pin digital distinto del Arduino mediante su correspondiente resistencia limitadora de corriente. En mi montaje se ha utilizado la siguiente asignación:

- LED **rojo** conectado al pin digital **13**.
- LED **naranja** conectado al pin digital **12**.
- LED **verde** conectado al pin digital **11**.

El terminal positivo (**ánodo**) de cada LED está conectado al **pin** digital correspondiente a través de una **resistencia**, mientras que el terminal negativo (**cátodo**) está conectado a tierra común (**GND**). Se han respetado además los colores de cableado pedidos en el enunciado, utilizando rojo para 5V y negro para tierra.

Diseño



Serial Monitor

```
Traffic light: RED
Traffic light: GREEN
Traffic light: ORANGE
```

Código

```
const byte red_LED = 13;      // Red led connected to digital pin 13
const byte orange_LED = 12;   // Orange led connected to digital pin 12
const byte green_LED = 11;    // Green led connected to digital pin 11

void setup()
{
  Serial.begin(9600);          // Starts serial communication at 9600 baud rate

  pinMode(red_LED, OUTPUT);    // Configures red pin as output
  pinMode(orange_LED, OUTPUT); // Configures orange pin as output
  pinMode(green_LED, OUTPUT);  // Configures green pin as output
}

void loop()                    // runs on a loop
{
  digitalWrite(red_LED, HIGH); // Turns red led on
  digitalWrite(orange_LED, LOW); // Turns orange led off
  digitalWrite(green_LED, LOW); // Turns green led off
  Serial.println("Traffic light: RED"); // Prints state on the Serial Monitor
  delay(3000);                  // Waits 3000 ms (3 seconds)

  digitalWrite(red_LED, LOW);  // Turns red led off
  digitalWrite(orange_LED, LOW); // Turns orange led off
  digitalWrite(green_LED, HIGH); // Turns green led on
  Serial.println("Traffic light: GREEN"); // Prints current state on SM
  delay(3000);                  // Waits 3000 ms (3 seconds)

  digitalWrite(red_LED, LOW);  // Turns red led off
  digitalWrite(orange_LED, HIGH); // Turns orange led on
  digitalWrite(green_LED, LOW); // Turns green led off
  Serial.println("Traffic light: ORANGE"); // Prints current state on SM
  delay(500);                   // Waits 500 ms (0.5 seconds)
}
```

Primero se declaran tres constantes de tipo byte para guardar los pines a los que están conectados los LED rojo, naranja y verde. Después, en *'setup ()'*, se inicia la comunicación serie a 9600 baudios y se configuran los tres pines como salidas. En *'loop ()'* se ejecutará indefinidamente la secuencia del semáforo. Para cada estado, se enciende únicamente el LED correspondiente usando *'digitalWrite (... , HIGH)'* y se apagan los otros dos con *'digitalWrite (... , LOW)'*. A continuación, se muestra en el monitor serie el estado actual del semáforo y se mantiene ese estado durante el tiempo establecido utilizando *'delay ()'*. Al finalizar la última fase, el ciclo se reinicia automáticamente.

Práctica 2. Control de acceso con Keypad 4x4

En este ejercicio realizaremos un circuito capaz de controlar el acceso de entrada a una puerta. Con un teclado 4x4 tendremos la posibilidad de almacenar cualquier contraseña tecleada, que será procesada y comparada con una contraseña registrada válida, para esta tarea, utilizaremos una contraseña de 4 números, que será: 1234

Descripción

En esta práctica se ha diseñado un sistema de control de acceso mediante un teclado 4x4. El usuario debe introducir una contraseña de 4 dígitos, que se compara con una contraseña válida previamente definida (“1234”). Si la contraseña es correcta, se concede acceso y se enciende un LED verde. Si la contraseña es incorrecta, se deniega el acceso y un LED rojo parpadea.

¿Cuál es el funcionamiento del teclado 4x4?

El teclado 4x4 está compuesto por 4 filas y 4 columnas, de ahí el nombre. Cada tecla representa la conexión entre una fila y una columna concreta. Cuando se pulsa una tecla, el Arduino detecta qué fila y columna están conectadas y así identifica el carácter pulsado.

El teclado se conecta mediante 8 pines: 4 correspondientes a las filas y otros 4 correspondientes a las columnas.

El microcontrolador escanea continuamente estas conexiones para detectar las pulsaciones del usuario.

¿Cómo se lee el dato y cómo se compara?

Cada vez que se pulsa una tecla, el valor se almacena en un array. A medida que se introducen los dígitos, se comparan con la contraseña válida almacenada.

Una vez introducidos los 4 caracteres:

- Si coinciden → se concede acceso (LED verde)
- Si no coinciden → se deniega acceso (LED rojo)

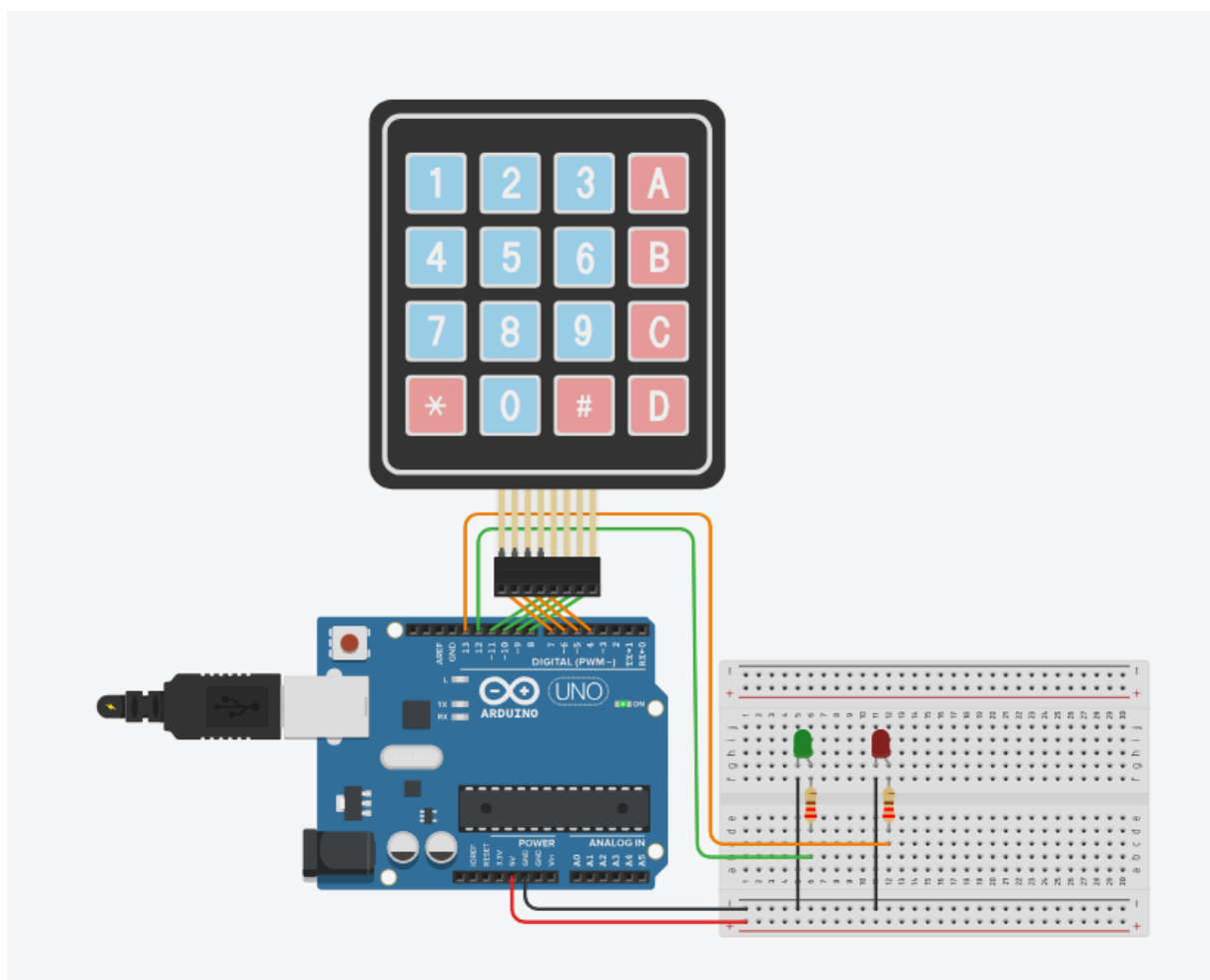
Modificación del código I: Sistema de bloqueo

Se ha añadido un contador de intentos fallidos. Si el usuario introduce una contraseña incorrecta tres veces consecutivas, el sistema se bloquea durante 10 segundos, impidiendo nuevas entradas.

```
int failedAttempts = 0; // Counter for failed password attempts

void loop()
{
  /****Check if system should be locked after 3 failed attempts***/
  if (failedAttempts >= 3)
  {
    Serial.println("SYSTEM BLOCKED - WAIT 10 SECONDS"); // Display block message
    delay(10000); // Blocks system for 10 seconds
    failedAttempts = 0; // Reset counter
  }
  /**** *****/
}
```

Modificación del diseño I: Filas y columnas cambiadas



Estas líneas de código

```
byte rowPins[ROWS]={11,10,9,8}; // Digital pins connected to keypad rows
byte colPins[COLS]={7,6,5,4}; // Digital pins connected to keypad columns
```

cambian a

```
byte rowPins[ROWS]={7,6,5,4}; // Digital pins connected to keypad rows
byte colPins[COLS]={11,10,9,8}; // Digital pins connected to keypad columns
```

Modificación del código II: Serial monitor

Se ha añadido el uso del monitor serie para mostrar: la tecla pulsada en cada momento y una serie de mensajes indicando si el acceso es concedido, denegado o bloqueado. Facilitando así la depuración y seguimiento del sistema.

```
void setup()
{
    Serial.begin(9600); // Initialize serial communication

    Serial.print("Key pressed: ");
    Serial.println(key);

    Serial.println("SYSTEM BLOCKED - WAIT 10 SECONDS");

    Serial.println("ACCESS GRANTED");

    Serial.println("ACCESS DENIED");
}
```

Código completo

```
1 //***** KEYPAD CONFIGURATION *****
2 #include <Keypad.h> // Library to handle keypad input
3
4 const byte ROWS = 4; // Number of rows in keypad
5 const byte COLS = 4; // Number of columns in keypad
6
7 // LED pins
8 const byte greenLED = 12; // Green led to pin 12
9 const byte redLED = 13; // Red led to pin 13
10
11 // Keypad character matrix
12 char keys[ROWS][COLS] =
13 {
14 { '1', '2', '3', 'A' },
15 { '4', '5', '6', 'B' },
16 { '7', '8', '9', 'C' },
17 { '*', '0', '#', 'D' },
18 };
19
20 byte rowPins[ROWS]={7,6,5,4}; // Digital pins connected to keypad rows
21 byte colPins[COLS]={11,10,9,8}; // Digital pins connected to keypad columns
22
23 // Create keypad object and set parameters
24 Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);
25 //***** END KEYPAD CONFIGURATION *****
26
27 // Variables for password handling
28 int i = 0; // Index for storing entered digits
29 int failedAttempts = 0; // Counter for failed password attempts
30
31 char valid[5] = {'1','2','3','4','\0'}; // Correct password
32 char pass[5] = {' ',' ',' ',' ','\0'}; // Array to store user input
33
34 void setup()
35 {
36     Serial.begin(9600); // Initialize serial communication
37     pinMode(greenLED, OUTPUT); // Set green LED pin as output
38     pinMode(redLED, OUTPUT); // Set red LED pin as output
39     digitalWrite(greenLED, LOW); // Green LED off
40     digitalWrite(redLED, LOW); // Red LED off
41 }
42
```

```

43 void loop()
44 {
45     //****Check if system should be locked after 3 failed attempts****
46     if (failedAttempts >= 3)
47     {
48         Serial.println("SYSTEM BLOCKED - WAIT 10 SECONDS"); // Display block message
49         delay(10000); // Blocks system for 10 seconds
50         failedAttempts = 0; // Reset counter
51     }
52     //**** ****
53     char key = keypad.getKey(); // Pressed key will be stored in char key
54     if (key != NO_KEY) // If key pressed:
55     {
56         Serial.print("Key pressed: "); // 1. Show pressed key
57         Serial.println(key); //
58         pass[i] = key; // 2. Store the pressed keys along the 'pass' array
59         i++; // 3. Move to next position
60         if (i == 4) // 4. If 4 digits are entered
61         {
62             i = 0; // - Reset index for next attempt
63             if (strcmp(pass, valid, 4) == 0) // - If pass array = valid array
64             {
65                 Serial.println("ACCESS GRANTED"); // - Display 'access granted' message
66                 digitalWrite(greenLED, HIGH); // - Turn on green LED
67                 delay(2000); // - Keep led on for 2 seconds
68                 digitalWrite(greenLED, LOW); // - Turn off green LED
69                 failedAttempts = 0; // - Reset failed attempts counter
70             }
71             else // Otherwise,
72             {
73                 Serial.println("ACCESS DENIED"); // - Display 'access denied' message
74                 failedAttempts++; // - Increase failed attempt counter
75                 for (int j = 0; j < 2; j++) // - LOOP: Blink red LED twice (300 ms)
76                 {
77                     digitalWrite(redLED, HIGH);
78                     delay(300);
79                     digitalWrite(redLED, LOW);
80                     delay(300);
81                 }
82             }
83         }
84     }
85 }

```

 Serial Monitor

```

Key pressed: 2
ACCESS DENIED
SYSTEM BLOCKED - WAIT 10 SECONDS
Key pressed: 1
Key pressed: 2
Key pressed: 3

```

Práctica 3. Detector de presencia con alarma sonora y visual

En este ejercicio crearemos un circuito capaz de detectar cualquier movimiento dentro de un área específica, que se muestra en verde claro una vez iniciada la simulación. El detector verificará cualquier movimiento interno, continuamente, y enviará dos posibles valores lógicos al pin digital 8: '0' No se detectó movimiento y '1' Movimiento detectado. En este segundo caso, una alarma estará activa continuamente hasta que alguien presione el botón de parada.

Descripción

En esta práctica se ha diseñado un sistema capaz de detectar movimiento dentro de un área mediante un sensor PIR. Cuando se detecta presencia, se activa una alarma compuesta por un buzzer y un LED rojo. La alarma permanece activa hasta que se presiona un botón de parada.

¿Cómo funciona el pulsador?

El **pulsador** es un componente digital que **permite abrir o cerrar un circuito** eléctrico **manualmente**. En este sistema lo utilizamos para detener la alarma una vez ha sido activada. Cuando el **pulsador no está presionado**, el pin de entrada recibe un valor lógico bajo (**LOW**). Al **presionar el botón**, el circuito se cierra y el pin recibe un valor lógico alto (**HIGH**).

En esta práctica, el pulsador está configurado para generar una interrupción mediante la instrucción '*attachInterrupt*', lo que permite ejecutar una función específica, en nuestro caso '*stopAlarm*', en el momento exacto en que se detecta un flanco de subida, es decir, un cambio de LOW a HIGH. Esto permite detener la alarma de forma inmediata sin necesidad de esperar al ciclo del programa principal.

¿Cómo funciona el buzzer?

El **buzzer** es un dispositivo electrónico que **convierte señales eléctricas en sonido**. En este circuito lo utilizamos para generar una alarma sonora cuando se detecta movimiento.

El buzzer funciona mediante la función **'tone'**, que **permite generar una señal de una frecuencia determinada durante un tiempo específico**. En este programa se usa para emitir tonos de distinta frecuencia. La frecuencia se mide en hercios (Hz), y determina el tono del sonido. En esta práctica utilizamos dos configuraciones: Un sonido de **600 Hz durante 0.6 segundos** y un sonido de **300 Hz durante 0.2 segundos**.

```
tone(pin_buzzer, 300, 200); // buzzer pin (9), 300Hz, during 0.2s
```

Estos dos sonidos se alternan mientras la alarma está activa, generando un patrón intermitente que facilita la identificación de la alerta.

¿Cómo funciona el sensor?

El **sensor PIR** detecta **cambios en la radiación infrarroja** del entorno. Esto quiere decir que, cuando se produce movimiento, el sensor envía un valor lógico de '1'. Si no hay movimiento, envía '0'.

Integración del pulsador y buzzer en el sistema

El buzzer y el pulsador trabajan conjuntamente para gestionar el estado de la alarma. Cuando el **sensor PIR** detecta movimiento, se **activa** la variable de alarma y el **buzzer** comienza a emitir señales sonoras de forma intermitente junto con el LED.

El pulsador permite desactivar este estado mediante una interrupción. Al presionarlo, se ejecuta la función que apaga tanto el buzzer como el LED y reinicia la variable de alarma.

Este uso combinado permite crear el sistema de alarma de forma que sea interactivo, la activación es automática y la desactivación manual.

Modificación del diseño y código: Motor de vibración

○ Modificación del montaje

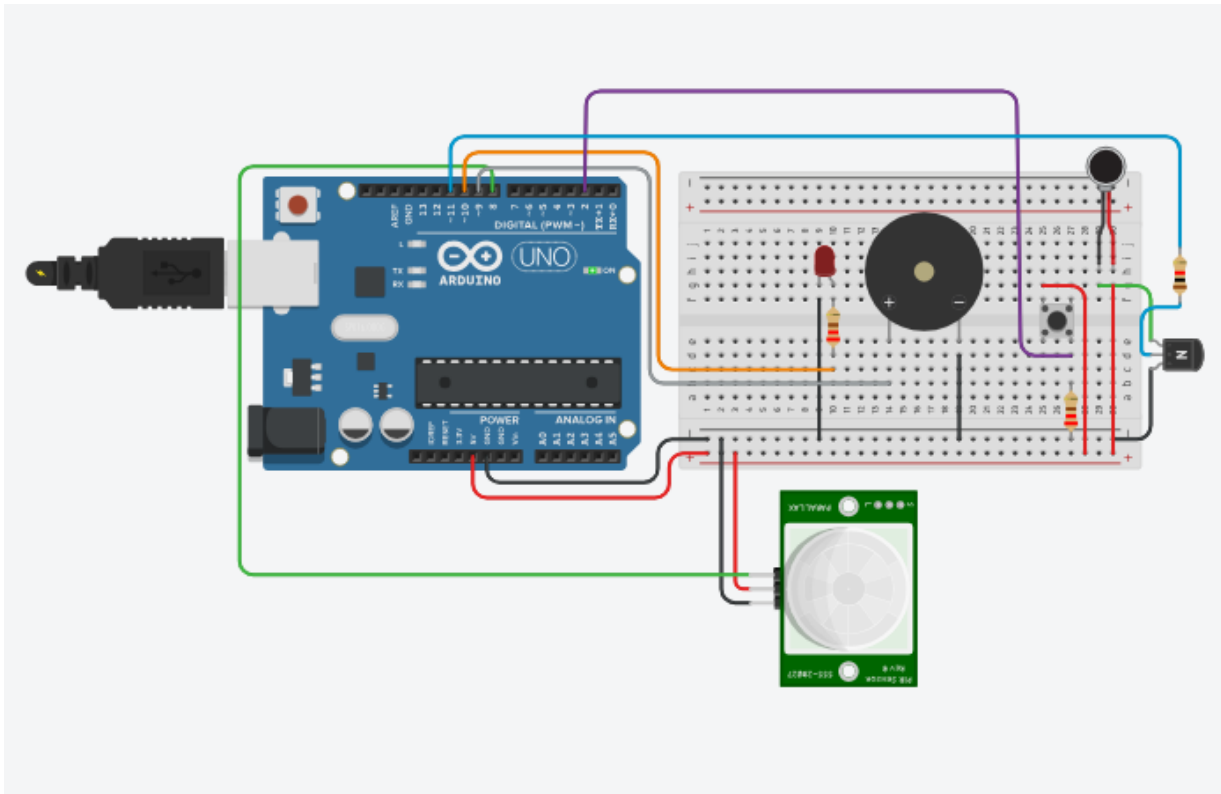
Aunque el enunciado indica que el motor de vibración puede conectarse directamente entre un pin digital y GND, en la práctica eso no es recomendable y nos dará un **WARNING** en Tinkercad. Esto es porque el motor es un dispositivo que consume más corriente que la que puede suministrar directamente un pin del Arduino (+20 mA) y si se conecta directamente, puede provocar un mal funcionamiento o dañar el microcontrolador.

Por este motivo, para el montaje hemos utilizado un **transistor** como elemento intermedio. El transistor actuará como un **interruptor** electrónico que nos permitirá controlar el motor utilizando la señal del Arduino sin que este tenga que suministrar directamente la corriente necesaria.

Para este diseño, hemos conectado el **pin digital** del Arduino a la **base del transistor** a través de una **resistencia**. Esta resistencia es necesaria para limitar la corriente que entra en la base del transistor y proteger tanto el transistor como el pin del Arduino.

Cuando el **Arduino** envía un **nivel alto** (HIGH) al pin, el **transistor** se **activa** y **permite el paso** de corriente entre el **colector** y el **emisor**, haciendo que el motor funcione. **Cuando** el pin está en **nivel bajo** (LOW), el **transistor** se **desactiva** y el motor se detiene.

De este modo, el motor se alimenta directamente desde la fuente de alimentación de 5V, mientras que el Arduino únicamente actúa como señal de control.



○ Modificación código

Se ha añadido al circuito un motor de vibración para que, cuando se active la alarma por detección de movimiento, también se genere una señal táctil además de la señal visual y sonora.

Para modificar nuestro código, primero se añade una nueva constante llamada '*pin_motor*' y se asigna al pin digital 11, que será el encargado de controlar el motor de vibración. Lo configuramos como salida mediante '*pinMode*'.

En nuestra función bucle principal, el programa lee el estado del sensor PIR. **Si el sensor detecta movimiento**, el sistema entra en modo alarma y se **activa el motor vibración** poniendo su pin en HIGH. Después, el programa genera un segundo tono de 300 Hz durante 200 ms, apaga el LED y apaga también el motor. Esta secuencia se repite continuamente mientras la alarma siga activa, siguiendo el patrón descrito anteriormente.

```

1  int input = 0;
2  int alarm = 0;
3
4  const byte pin_led = 10;
5  const byte pin_sensor = 8;
6  const byte pin_buzzer = 9;
7  const byte pin_button = 2;
8  const byte pin_motor = 11;           // Define motor pin (11)
9
10
11 void setup()
12 {
13     Serial.begin(9600);               // Serial port speed: 9600 symbols per second
14
15     pinMode(pin_sensor, INPUT);
16     pinMode(pin_buzzer, OUTPUT);
17     pinMode(pin_led, OUTPUT);
18     pinMode(pin_button, INPUT);
19     pinMode(pin_motor, OUTPUT);
20
21     digitalWrite(pin_led, LOW);       // LED initially 'off'
22     digitalWrite(pin_buzzer, LOW);    // Buzzer initially 'off'
23     digitalWrite(pin_motor, LOW);     // Motor initially 'off'
24
25     attachInterrupt(0, stopAlarm, RISING); // Interrupt on button press
26 }
27
28 void loop()
29 {
30     input = digitalRead(pin_sensor);   // Store digital input from pin 8 in 'input'
31                                         // '0' NO MOTION DETECTED - '1' MOTION DETECTED
32     Serial.println(input);
33
34     if (input == HIGH)                 // If motion detected...
35         alarm = 1;                    // Alarm acticated
36
37     Serial.print("alarm: ");
38     Serial.println(alarm);
39
40     if (alarm == 1)
41     {
42         tone(pin_buzzer, 600, 600);    // This function defines parameters to the buzzer
43                                         // pin 9, buzz 600 Hz, during 0.6s
44         digitalWrite(pin_led, HIGH);    // Led 'on'
45         digitalWrite(pin_motor, HIGH);  // Motor 'on'
46         delay(600);                    // Keep instructions active for 0.6s
47
48         tone(pin_buzzer, 300, 200);     // pin 9, buzz 300Hz, during 0.2s
49         digitalWrite(pin_led, LOW);     // Led 'off'
50         digitalWrite(pin_motor, LOW);   // Motor 'off'
51         delay(200);                    // Kepp instructions active for 0.2s
52     }
53 }
54
55 void stopAlarm()
56 {
57     digitalWrite(pin_buzzer, LOW);     // Buzzer 'off'
58     digitalWrite(pin_led, LOW);        // Led 'off'
59     digitalWrite(pin_motor, LOW);      // Motor 'off'
60     alarm = 0;                         // Reset alarm
61 }

```



Serial Monitor

```

0
alarm: 0
0
alarm: 0
1
alarm: 1
1
alarm: 1
0
alarm: 1
0
alarm: 1
0
alarm: 1

```

Práctica 4. Detector de obstáculos con alarma intermitente

En este nuevo proyecto crearemos un Esquema, capaz de detectar cualquier obstáculo dentro del área cubierta por el detector ultrasónico. La distancia entre el detector y el obstáculo se calculará y registrará, y dependiendo de esta longitud, los pitidos emitidos por el timbre se repetirán más rápido, acercándose la distancia entre ellos

Descripción

En esta práctica se ha diseñado un sistema capaz de detectar la distancia a un obstáculo mediante un sensor ultrasónico. En función de dicha distancia, el sistema genera una alarma sonora mediante un buzzer y una señal visual mediante un LED RGB, variando tanto la frecuencia del sonido como el color del LED según la proximidad del objeto.

¿Cómo funciona el sensor ultrasónico?

El **sensor ultrasónico** permite medir la distancia a un objeto mediante la **emisión** de **ondas sonoras de alta frecuencia**. Su funcionamiento se basa en el envío de un **pulso ultrasónico** y la medición del **tiempo que tarda** en regresar **tras** reflejarse en un **obstáculo**. El proceso es el siguiente:

Primero, el sensor emite un pulso ultrasónico (*trigger*). A continuación, la onda se propaga por el aire hasta chocar con un objeto. La señal rebota y vuelve al sensor, donde se mide el tiempo de ida y vuelta del pulso.

A partir de este tiempo, se calcula la distancia utilizando la velocidad del sonido (aproximadamente **340 m/s / 0,034 cm/μs**). En el código, este cálculo se realiza mediante la función '*pulseIn*', que detecta cuánto tiempo la señal permanece en estado alto.

La fórmula utilizada es:

$$\text{distancia} = \text{tiempo} / 29 / 2$$

Si hacemos el cálculo, 1 **cm tarda** aproximadamente **29 μs** en recorrerse. Por eso, '*tiempo / 29*' convierte el tiempo (microsegundos) a distancia (centímetros) y '*/ 2*' nos corrige

para quedarse solo con el recorrido de ida de la onda. Así, se nos permite determinar la proximidad del objeto y ajustar el comportamiento del sistema.

¿Cómo funcionan los LED RGB?

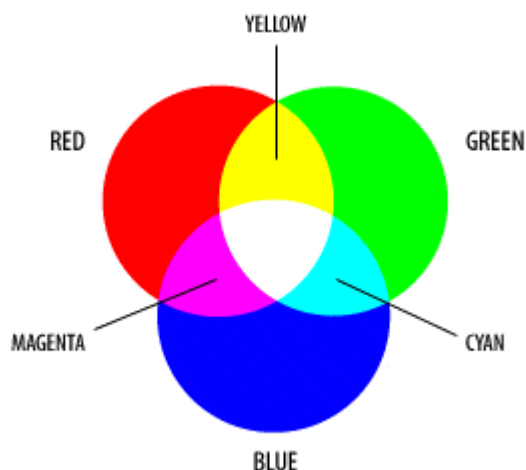
El **LED RGB** es un componente que **contiene tres LED en su interior: rojo (R), verde (G) y azul (B)**. Combinando diferentes intensidades de estos tres colores se puede generar una amplia variedad de colores.

En esta práctica, queremos utilizar el LED RGB para representar visualmente la distancia al obstáculo. Cuando el objeto se encuentra **lejos**, el LED se iluminará en color verde. A medida que la distancia disminuye, el color irá cambiando progresivamente, pasando a amarillo para una **distancia media** y a naranja cuando el objeto esté más **cercano**. Si el objeto se encuentra **muy próximo**, el LED mostrará el color magenta, y finalmente, cuando exista **peligro de colisión**, el LED se encenderá en color rojo.

En nuestro código, utilizamos instrucciones '*digitalWrite*' para encender o apagar cada uno de los pines del LED. Cada color se obtendrá activando diferentes combinaciones de los pines. Por ejemplo:

- **Verde** → pin **verde** en **HIGH**
- **Amarillo** → pin **rojo** y pin **verde** en **HIGH**

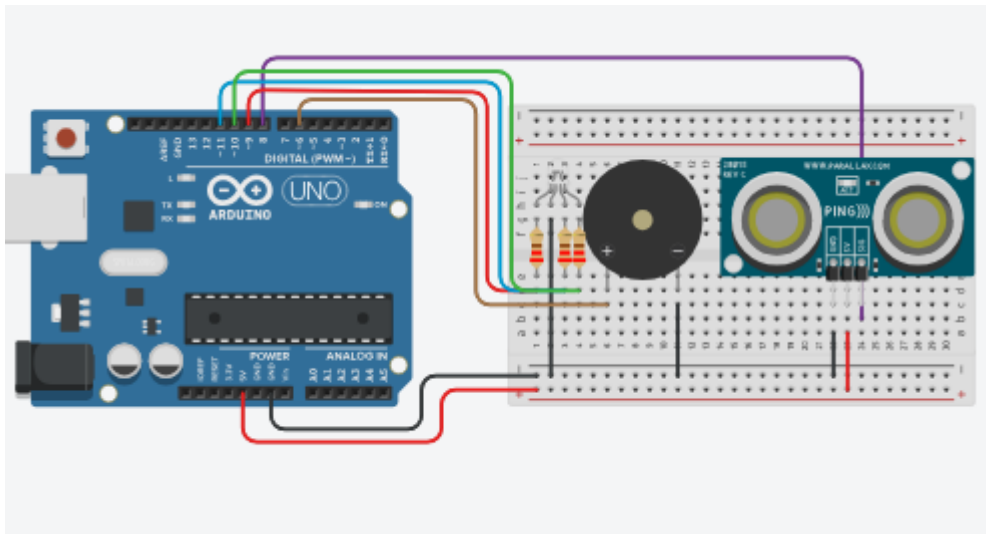
Es importante tener en cuenta que **cada canal del LED** debe llevar una **resistencia** para limitar la corriente y evitar daños.



	RGB (255, 0, 0)
	RGB (255, 127, 0)
	RGB (255, 255, 0)
	RGB (0, 255, 0)
	RGB (0, 0, 255)
	RGB (75, 0, 130)
	RGB (143, 0, 255)

Modificación de código: LED RGB

En el montaje, el LED RGB dispone de cuatro patas. Tres de ellas corresponden a los canales de color rojo, verde y azul, mientras que la cuarta es la pata **común**. En este caso, se utiliza un LED RGB de **cátodo común**, por lo que dicha pata común se conecta a **GND**. Cada uno de los pines de color se conecta a un pin digital del Arduino a través de una resistencia para limitar la corriente.



En el siguiente código modificado hemos sustituido el LED simple por un LED RGB para representar visualmente la distancia detectada por el sensor ultrasónico.

El código tiene el mismo funcionamiento que antes. En primer lugar, se genera un pulso en el sensor y se mide el tiempo que tarda en regresar la señal reflejada mediante la función *'pulseIn'*. A partir de ese tiempo, se calcula la distancia en centímetros e inches y se muestra por el Serial Monitor. Después, el programa compara esa distancia con varios rangos previamente definidos y, en función de ellos, activa un color distinto en el LED RGB además de modificar el comportamiento del buzzer. Para organizar mejor el programa, se han cambiado las funciones independientes (*func1*, *func2*...) para definir cada color (*set_green*, *set_red*...).

En el caso del color naranja, no es suficiente con utilizar únicamente *'digitalWrite'*, ya que esa función solo permite encender o apagar completamente cada canal del LED RGB. Si se activan el rojo y el verde al máximo, el resultado visual se aproximará más al amarillo que al naranja. Por ello, para obtener un color naranja he utilizado la función *'analogWrite'*, que permite aplicar PWM y controlar la intensidad de cada canal (0 - 255). De esta manera, se genera el color combinando una intensidad alta en el canal rojo y una intensidad media-baja en el canal verde.

```

1  const byte pin_sensor = 8;
2  const byte pin_buzzer = 6;
3
4  // RGB LED pins
5  const byte pin_red = 9;
6  const byte pin_green = 10;
7  const byte pin_blue = 11;
8
9  void setup()
10 {
11   Serial.begin(9600);           // Serial port speed: 9600 symbols per second
12
13   pinMode(pin_buzzer, OUTPUT);  // Buffer pin
14
15   pinMode(pin_red, OUTPUT);     // RED Led pin
16   pinMode(pin_green, OUTPUT);   // GREEN Led pin
17   pinMode(pin_blue, OUTPUT);    // BLUE Led pin
18 }
19
20 /*
21  * // Sets RGB color using ON/OFF (digital signals)
22  *void setColor(bool r, bool g, bool b)
23  *{
24  *   digitalWrite(pin_red, r);
25  *   digitalWrite(pin_green, g);
26  *   digitalWrite(pin_blue, b);
27  *}
28 */
29
30 // Sets RGB color using PWM intensity values (0-255)
31 void setColor_pwm(int r, int g, int b)
32 {
33   analogWrite(pin_red, r);
34   analogWrite(pin_green, g);
35   analogWrite(pin_blue, b);
36 }
37
38 void loop()
39 {
40   float duration, inch, cm;
41
42   pinMode(pin_sensor, OUTPUT);    // Creating ping pulse 'LOW' 2us, 'HIGH' 5us
43   digitalWrite(pin_sensor, LOW);
44   delayMicroseconds(2);
45   digitalWrite(pin_sensor, HIGH);
46   delayMicroseconds(5);
47   digitalWrite(pin_sensor, LOW);
48
49   pinMode(pin_sensor, INPUT);     // After sampling ping pulse, pin 8 as input
50   duration = pulseIn(pin_sensor, HIGH); // Response time for getting back the ping pulse [microsec]
51
52   cm = duration / 29 / 2;         // Sound speed: 340 m/s or 29microsec/cm

```

```

53  inch = duration / 74 / 2;           // Sound speed: 13385.83 inches/s or 74 microsec/inch
54
55  Serial.print(duration);
56  Serial.println(" microseconds");
57  Serial.print(inch);
58  Serial.print(" in");
59  Serial.println();
60  Serial.print(cm);
61  Serial.println(" cm");
62  Serial.println();
63
64  delay(100);
65
66  if(cm<=300 && cm>200)                // Range 3m - 2m [GREEN]
67    set_green();
68  else if(cm<=200 && cm>100)           // Range 2m - 2m [YELLOW]
69    set_yellow();
70  else if(cm<=100 && cm>50)            // Range 1m - 0.5m [ORANGE]
71    set_orange();
72  else if(cm<=50 && cm>25)             // Range 0.5m - 0.25m [MAGENTA]
73    set_magenta();
74  else if(cm<=25)                    // Range < 0.25m (continuous beep) [RED]
75    set_red();
76  else                                // Out of range(no beep) [no color]
77  {
78    //setColor(LOW, LOW, LOW);
79    setColor_pwm(0, 0, 0);
80    noTone(pin_buzzer);
81  }
82
83 }
84 void set_green()
85 {
86   //setColor(LOW, HIGH, LOW);
87   setColor_pwm(0, 255, 0);
88   tone(pin_buzzer, 400, 100);
89   delay(300);
90 }
91 void set_yellow()
92 {
93   //setColor(HIGH, HIGH, LOW);
94   setColor_pwm(255, 255, 0);
95   tone(pin_buzzer, 400, 100);
96   delay(200);
97 }
98 void set_orange()
99 {
100   //setColor(HIGH, LOW, LOW); // Orange = Red
101   setColor_pwm(255, 20, 0);    // Orange: max red + low green
102   tone(pin_buzzer, 400, 100);
103   delay(100);
104 }
105 void set_magenta()
106 {
107   //setColor(HIGH, LOW, HIGH);
108   setColor_pwm(255, LOW, 255);
109   tone(pin_buzzer, 400, 100);
110   delay(50);
111 }
112 void set_red()
113 {
114   //setColor(HIGH, LOW, LOW);
115   setColor_pwm(255, 0, 0);
116   tone(pin_buzzer, 400);
117 }
118

```



Serial Monitor

```

4276.00 microseconds
28.89 in
73.72 cm

```

```

4276.00 microseconds
28.89 in

```

Práctica 5. Detector de gas con alarma

En este proyecto crearemos un diseño que nos proporcionará información sobre la calidad del aire, pudiendo detectar gases inflamables, humo y otros gases peligrosos. Estableceremos 4 rangos de medición diferentes, siendo el primero la situación de mayor seguridad, subiendo a un led verde y pasando por el siguiente con un led amarillo, y el siguiente con un led naranja que se muestra, finalmente llegamos al máximo situación peligrosa donde el gas peligroso presenta su densidad máxima, en este caso sonará una alarma y se mostrará un led rojo.

Descripción

En esta práctica se ha diseñado un sistema capaz de detectar la presencia de gases inflamables, humo y otros gases peligrosos mediante un sensor analógico MQ2. En función del valor leído por el sensor, el sistema indica el nivel de peligro mediante cuatro LED de distinto color y activa una alarma sonora con un buzzer en los niveles más altos. Además, como modificación, hemos añadido un microservo que controla una trampilla de ventilación, variando su apertura según el nivel de gas detectado.

¿Cómo funciona un sensor de gas?

El sensor de gas MQ2 es un **sensor analógico** capaz de detectar gas licuado del petróleo, butano, propano, metano, alcohol, hidrógeno, humo y otros gases inflamables. Su funcionamiento se basa en la **variación de su tensión de salida según la concentración de gas** presente en el ambiente. Cuando la concentración de **gas aumenta**, el sensor entrega una **señal analógica mayor**; en cambio, cuando el aire está limpio, la señal de salida es menor.

El enunciado indica que, para esta práctica, se consideren valores de entrada comprendidos entre **590** y **921**, que corresponden aproximadamente a tensiones desde **2,88 V** hasta **4,50 V**. Para comprobar estos valores se utiliza un **multímetro** en el montaje. A partir de estas mediciones, se definen varias zonas de peligro.

En cuanto a su uso, el sensor MQ2 se emplea **para monitorizar la calidad del aire y detectar situaciones peligrosas**. En esta práctica, lo utilizamos para clasificar el nivel de riesgo en cuatro estados. Si el valor de entrada es igual o menor que 590, se considera ausencia de contaminantes y se enciende el LED verde. Entre 591 y 699 se activa el LED amarillo. Entre 700 y 809 se activa el LED naranja y el buzzer comienza a sonar. A partir de 810 se enciende el LED rojo y el sistema entra en la situación de mayor peligro.

Desde el punto de vista de la **programación**, el **sensor** se conecta al **pin analógico A0** del Arduino y se lee mediante la función '*analogRead(A0)*'. El valor leído se almacena en una variable y después se compara con los distintos rangos usando estructuras '*else if*'. De esta forma, el sensor actúa como elemento principal de entrada del sistema, mientras que el resto de componentes funcionan como salida de aviso y/o respuesta.

¿Cómo funciona un microservo?

El **microservo** es un **actuador** que permite **posicionar su eje en un ángulo concreto**. A diferencia de un motor normal, no gira libremente de manera continua, sino que se mueve hasta la posición indicada y la mantiene.

En esta práctica se utiliza para simular una trampilla de ventilación que se abre en mayor o menor medida según el nivel de gas detectado. Su uso consiste en **transformar** una **señal eléctrica** enviada por el Arduino **en una posición angular** concreta. Así, cuando el nivel de gas es moderado, el servo abre ligeramente la trampilla; cuando el nivel de gas es más alto, la abre más; y cuando la situación es crítica, la abre completamente.

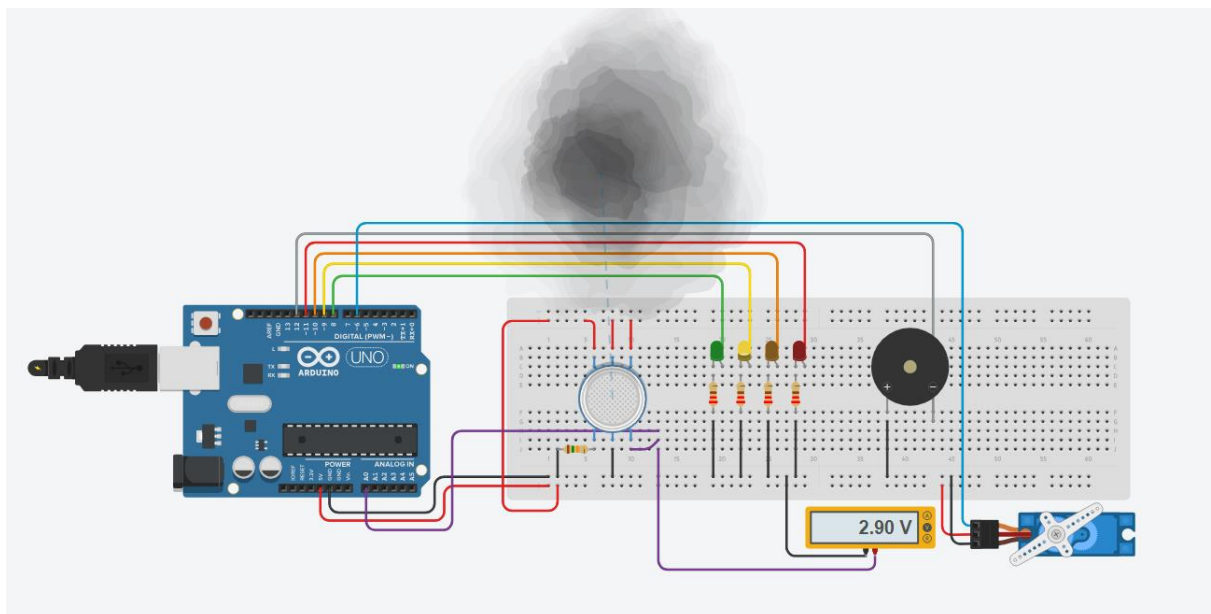
En **programación**, el microservo se controla mediante la librería '*Servo.h*'. Primero crearemos un **objeto de tipo Servo**, después indicaremos el pin al que está conectado con la función '*attach*' y finalmente fijaremos la posición deseada mediante la función '*write*'.

Modificación de código: Microservo

En nuestro nuevo código empezamos por importar la **librería ‘Servo.h’**, necesaria para controlar el microservo. Después **declaramos** los **pines** de los **LED**, del **buzzer** y del **servo**, así como una **variable input** que almacene el valor leído por el **sensor**.

En **‘setup’** iniciamos la **comunicación serie a 9600 baudios** y configuramos los LED y el buzzer como salidas. A continuación, asociamos el servo al pin digital 6 mediante **‘microservo.attach(pin_servo)’** y lo colocamos a una **posición inicial de 0 grados**.

En **‘loop’** leemos continuamente el valor del sensor con la función **‘analogRead(A0)’**. Ese valor lo imprimiremos en el Serial Monitor para facilitar la observación del circuito. Después, el programa evaluará el valor de entrada mediante varias **condiciones**: si el valor es **menor o igual que 590**, se enciende el LED verde, no suena el buzzer y el servo permanece a 0°; si el valor está **entre 591 y 699**, se enciende el LED amarillo y el servo pasa a 30°; si el valor está **entre 700 y 809**, se enciende el LED naranja, el buzzer suena continuamente a 600 Hz y el servo pasa a 90°; si el valor es **igual o mayor que 810**, se enciende el LED rojo, el buzzer cambia a 300 Hz y el servo se coloca en 180°.



```

1 #include <Servo.h> // Library including servo obj and functions
2
3 const byte green_led = 8;
4 const byte yellow_led = 9;
5 const byte orange_led = 10;
6 const byte red_led = 11;
7
8 const byte pin_buzzer = 12;
9 const byte pin_servo = 6;
10
11 int input = 0; // Analog value will be stored here
12
13 Servo microservo; // Declare object microservo
14
15 void setup()
16 {
17     Serial.begin(9600); // Serial port speed: 9600 symbols per second
18
19     pinMode(green_led, OUTPUT); // Green led
20     pinMode(yellow_led, OUTPUT); // Yellow led
21     pinMode(orange_led, OUTPUT); // Orange led
22     pinMode(red_led, OUTPUT); // Red led
23     pinMode(pin_buzzer, OUTPUT); // Buzzer
24
25     microservo.attach(pin_servo); // Attach servo to pin 6
26     microservo.write(0); // Initial servo position
27 }
28 void loop()
29 {
30     input = analogRead(A0); // Input value
31     Serial.println(input); // Show this value on Serial Monitor
32
33     if (input < 591) // 1st range situation. The lowest density
34     {
35         digitalWrite(green_led, HIGH); // Green led 'on' and no tone
36         digitalWrite(yellow_led, LOW);
37         digitalWrite(orange_led, LOW);
38         digitalWrite(red_led, LOW);
39         noTone(pin_buzzer);
40         microservo.write(0); // Vent closed
41     }
42
43     else if (input >= 591 && input < 700) // 2nd range situation
44     {
45         digitalWrite(green_led, LOW);
46         digitalWrite(yellow_led, HIGH); // Yellow led 'on' and no tone
47         digitalWrite(orange_led, LOW);
48         digitalWrite(red_led, LOW);
49         noTone(pin_buzzer);
50         microservo.write(30); // Servo at 30 degrees
51     }
52
53     else if (input >= 700 && input < 810) // 3rd range situation
54     {
55         digitalWrite(green_led, LOW);
56         digitalWrite(yellow_led, LOW);
57         digitalWrite(orange_led, HIGH); // Orange led 'on'
58         digitalWrite(red_led, LOW);
59         tone(pin_buzzer, 600); // Buzzer beeps continuously
60         microservo.write(90); // Servo at 90 degrees
61     }
62
63     else if (input >= 810) // 4th range situation. The highest density
64     {
65         digitalWrite(green_led, LOW);
66         digitalWrite(yellow_led, LOW);
67         digitalWrite(orange_led, LOW);
68         digitalWrite(red_led, HIGH); // Red led 'on'
69         tone(pin_buzzer, 300); // Buzzer beeps continuously
70         microservo.write(180); // Servo at 180 degrees
71     }
72 }

```

 Serial Monitor

763
651
651
651
651
605
605